ChatGPT said:

13. Complete starter code — first working version

Let's begin with a small but functional application. This first version will:

- Let users select multiple medicines.
- Check every pair against a local, source-documented interaction dataset.
- Display the interaction description and source.
- Distinguish a known match from a pair with no matching record.
- Include a separate, clearly labeled educational demonstration record so you can test the interface before importing verified data.

We will use **Streamlit + Pandas + SQLite-ready CSV data** for the starter version. The initial dataset will contain no fabricated medical interaction claims. The demonstration row is explicitly fictional and is not used to produce medical warnings.

Step A — Set up your project

Open VS Code and create a folder named `medication-safety-checker`.

Open its terminal using **Terminal** → **New Terminal**.

On Windows, run:

Bash

```
py -m venv .venv
.venv\Scripts\activate
python -m pip install streamlit pandas requests
```

If `py` does not work, try `python` instead. If PowerShell blocks activation, open Command Prompt in VS Code or activate the environment using the appropriate terminal command.

Create these two files:

Plain text

```
medication-safety-checker/
├── app.py
└── interactions.csv
```

Step B — Create `interactions.csv`

Paste this into `interactions.csv`:

```csv
drug_a,drug_b,description,source_name,source_url,record_type
```

This is an empty dataset header. It deliberately contains no unverified interaction claims. The application will work and explain that its dataset has no records until you add verified data.

Step C — Create `app.py`

Copy the following complete code.

```python
import itertools
from pathlib import Path

import pandas as pd
import streamlit as st

DATA_FILE = Path(__file__).parent / "interactions.csv"

st.set_page_config(
    page_title="Medication Safety Checker",
    page_icon="💊",
    layout="centered",
)

st.title("💊 AI-Based Medication Safety Checker")

st.warning(
    "Educational prototype only. This application cannot determine "
    "whether a medicine combination is safe. It is not a substitute "
```

Step D — Run the application

In the terminal, execute:

```bash
```

```
streamlit run app.py
```

Your browser should open the application, usually at `http://localhost:8501`.

Because the initial CSV is empty, the app will show a message explaining that no verified interaction records have been loaded. **That is expected behavior, not an error.**

This is the safe foundation. Next, we will add real source-backed records, medicine search, and an API integration.

14. How to obtain real drug-label information

The safest approach is to separate **finding a drug label** from **confirming an interaction between two drugs**.

The official openFDA drug-label API ↗ returns public prescribing-label information. Its fields can vary between labels, and the API documentation cautions that the information should not be relied on for medical-care decisions.

Open FDA +2

Test the API before connecting it to your app

Create a file called `test_api.py`:

Python

```python
import requests

url = "https://api.fda.gov/drug/label.json"

params = {
    "search": 'openfda.generic_name:"caffeine"',
    "limit": 1,
}

try:
    response = requests.get(url, params=params, timeout=15)

    if response.status_code == 404:
        print("No matching label was found for this query.")
    else:
        response.raise_for_status()
        payload = response.json()
```

Run it:

```bash
python test_api.py
```

This is an API connectivity exercise, not a medical interaction test. Caffeine is simply a query example. The script retrieves a label, if one matches; it does not establish that caffeine interacts with any other medicine.

If no result is returned, try another search term or use the official API query guide ↗.

How to turn a label into a verified interaction record

1. Retrieve the original drug label.
2. Find its Drug Interactions section.
3. Identify the second medicine or substance discussed.
4. Confirm that the text supports the specific pair and the wording you plan to display.
5. Record the source URL and the date you checked it.
6. Add a structured pair only after the record has been reviewed against the source.

Do not automatically turn every paragraph in a label into a drug-pair warning. A label may describe an entire class of medicines, a mechanism, or an interaction that depends on the patient's circumstances.

15. How to add verified records to your CSV

Once you have confirmed a particular interaction from a reliable source, add a row using this format:

```csv
drug_a,drug_b,description,source_name,source_url,record_type
"Verified medicine A","Verified medicine B","Source-supported interaction description","Official dru
```

This is a **format example only**. The medicine names, description, and URL above are placeholders, not a real interaction record. Replace them with information you have actually verified before using the row.

Your current application deliberately ignores records that are not marked `verified`. This prevents an unreviewed demonstration record from becoming a medical warning.

After updating the CSV, restart the application or clear Streamlit's cache and rerun it.

16. How to make the project genuinely AI/ML-based

Once your basic checker works, implement these extensions in order:

1. **Medicine-name matching**

   Use RapidFuzz to suggest possible matches for misspelled names. Require the user to confirm the exact medicine instead of silently choosing a match.

2. **NLP-based interaction search**

   Use TF-IDF and cosine similarity to retrieve relevant passages from verified drug-label text. Show the original passage and source alongside the result.

3. **Optional ML classifier**

   If you obtain a sufficiently large, reliable, labeled dataset, train Logistic Regression or an SVM to classify interaction descriptions into defined categories. Evaluate it on unseen examples.

4. **Grounded explanation**

   Convert technical text into simpler language while retaining the original source. Do not allow an AI-generated explanation to create an interaction claim absent from the evidence.

17. What you should complete first

Use this checklist to track your progress.

Project progress 0 of 10

- [ ] Install Python and VS Code
- [ ] Create and activate the virtual environment
- [ ] Install Streamlit, Pandas and Requests
- [ ] Create interactions.csv
- [ ] Create app.py and run the application
- [ ] Test the openFDA API independently
- [ ] Collect and verify source-backed interaction records
- [ ] Connect the records to the checker
- [ ] Add medicine-name matching and NLP
- [ ] Test, evaluate and document limitations

My advice: First get the starter app running on your laptop. Then connect verified data before adding more AI features. You will have a much stronger project if you can demonstrate exactly where each medical interaction claim comes from.

For your report, describe the initial version honestly as a *source-backed medication interaction checker*, and describe the NLP or classification component separately as the AI/ML extension.